LAPORAN PRAKTIKUM
PEMROGRAMAN BERORIENTASI OBJEK
MODUL 3 – KONSEP DASAR PEMROGRAMAN BERORIENTASI OBJEK



NAMA: ANDI MUHAMMAD ABID JAYA

NIM: 103132400014

KELAS: S1-SD-05-01

PROGRAM STUDI SAINS DATA

FAKULTAS INFORMATIKA

TELKOM UNIVERSITY PURWOKERTO

2026

A. Teori

Dalam mudul ini mempelajari beberapa dasar teori dasar sebagai berikut:

1. Abstraksi (*Abstraction*): Menentukan ciri-ciri (atribut) dan perilaku (fungsi) penting suatu objek dengan mengabaikan detail insidental.
2. Enkapsulasi (*Encapsulation*): Pembungkusan data dan *method* untuk menyembunyikan rincian implementasi (*data hiding*) serta menjaga modularitas.
3. Pewarisan (*Inheritance*): Proses membuat kelas baru (*subclass*) dari kelas yang ada (*superclass*) dengan mewarisi sifat dan atributnya.
4. Polimorfisme (*Polymorphism*): Konsep "satu *interface* banyak aksi", yaitu menggunakan *interface* sama untuk perintah yang proses atau hasilnya dapat berbeda.
5. Kelas (*Class*) dan Objek (*Object*):
    a) Kelas: Cetak biru (*blueprint*) atau rancangan yang mendefinisikan atribut dan *method*.
    b) Objek: Instansiasi atau hasil ciptaan nyata dari suatu kelas.
6. Jenis Field (Atribut):
    a) Class Field: Variabel bermodifier static yang terikat pada kelas dan nilainya dipakai bersama oleh seluruh objek.
    b) Instance Field: Variabel tanpa modifier static di mana setiap objek memiliki salinan datanya masing-masing.
7. Method dan Pengiriman Parameter:
    a) Method: Fungsi (mengembalikan nilai dengan return) atau prosedur (tanpa nilai kembalian / void).
    b) Pass by Value: Digunakan pada tipe data primitif dengan menyalin isi nilai, sehingga perubahan di dalam *method* tidak mengubah variabel asal.

c) Pass by Reference: Digunakan pada tipe data objek dan array dengan mengirim alamat memori, sehingga perubahan di dalam *method* langsung mengubah data variabel asal.

8. Keyword this: Referensi khusus yang merujuk pada objek yang sedang aktif, digunakan untuk membedakan *instance field* dari parameter dengan nama sama atau memanggil konstruktor lain.

B. Guided

1. Enkapulasi

```
// nama file : Rekening.java

package guided;

public class Rekening {
   private int saldo = 0;

   public void tambahSaldo (int jumlah) {
      saldo = saldo + jumlah;
      System.out.println("saldo berhasil ditambahkan");
   }

      public void tampilkanSaldo() {
      System.out.println("saldo anda: " + saldo);
   }
}
```

```
// nama file: Enkapulasi.java

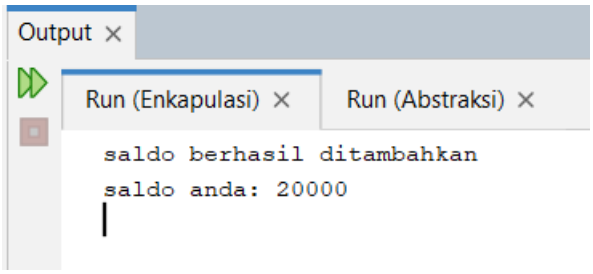
        package guided;

        public class Enkapulasi {
           public static  void main(String[] args) {
              Rekening rek = new Rekening();

              rek.tambahSaldo(20000);
              rek.tampilkanSaldo();
```

```
        }
}
```

Output :



2. Abstraksi

```
// nama file : kucing.java

package guided;

public class kucing {
    String nama;
    String ras;

    void bersuara () {
        System.out.println( nama + ras + "meow" );
    }
}
```

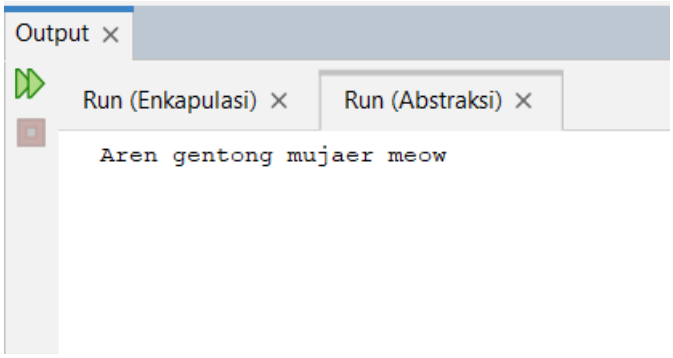
```
// nama file : Abstraksi.java

        package guided;

        public class Abstraksi {
            public static void main(String[] args){
                kucing aren = new kucing();

                aren.nama = "Aren gentong " ;
                aren.ras = "mujaer " ;
                aren.bersuara () ;
            }
}
```

Output :



C. Unguided

1. Soal Unguided 1

```
# Soal Unguided — Modul 3: Pemrograman Berorientasi Objek
### Penerapan OOP Sederhana untuk Pengolahan Data

---

## Petunjuk Umum

Buatlah program Java berdasarkan cerita berikut. Rancang sendiri struktur program yang diperlukan sebelum menuliskan
kode.

- Tentukan sendiri field, constructor, dan method yang diperlukan, mengikuti hint yang disediakan.
- Data di dalam object tidak boleh diakses atau diubah langsung dari luar class-nya.
- Gunakan nama parameter yang sama dengan nama field pada constructor (sehingga `this` perlu dipakai secara eksplisit).
- Program ini harus memuat dua jenis field: **instance field** (milik masing-masing object) dan **class field** (milik
class, dipakai bersama).
- Tidak perlu inheritance, overriding, maupun abstract class.

---

## Soal — Pembersihan dan Analisis Data Suhu Mingguan

Sebuah tim mengumpulkan suhu harian suatu kota selama satu minggu (7 hari) dari sensor otomatis. Data ini sudah
tersedia sebagai array di `main` sebelum diserahkan untuk diolah.

Pada salah satu hari, sensor gagal mengirim data. Hari yang gagal terekam ditandai dengan sebuah nilai penanda pada
array, karena suhu asli tidak mungkin sebesar itu. Nilai penanda ini sama untuk semua data suhu yang diolah kapan pun
dan oleh siapa pun — bukan sesuatu yang berbeda-beda tiap kali data dibuat — sehingga sebaiknya ditulis satu kali saja
sebagai aturan tetap milik class, bukan diketik ulang sebagai angka `-1.0` di setiap method yang membutuhkannya.

Buatlah satu class yang bertugas mengolah data suhu ini — memeriksa apakah ada data kosong, mengisinya, dan
menghasilkan statistik dasar dari data yang sudah bersih.


### Kebutuhan program

Class ini harus dapat:

1. **Menyimpan data suhu 7 hari**, dibuat dari array suhu mentah yang sudah ada di `main` (index array mewakili hari).
2. **Menyimpan nilai penanda data kosong (`-1.0`) sebagai aturan tetap milik class**, yaitu class field yang nilainya
tidak dapat diubah dan dipakai bersama oleh semua object.
3. **Menampilkan seluruh data apa adanya**, menandai hari yang kosong secara jelas.
4. **Mencari apakah ada hari yang kosong, dan di index berapa**, karena index inilah yang nanti dipakai untuk proses
pengisian data.
5. **Mengisi hari yang kosong**, menggunakan index dari poin 4, dengan nilai perkiraan berdasarkan rata-rata suhu satu
hari sebelum dan satu hari sesudahnya.
6. **Menghitung rata-rata suhu** dari data yang sudah bersih.

**Method yang disarankan:**

| Method | Kebutuhan | Return |
|---|---|---|
| `tampilkanData()` | 3 | `void` |
| `cariIndexKosong()` | 4 | `int` — index hari yang bernilai `NILAI_KOSONG`, atau `-1` apabila tidak ditemukan |
| `isiDataKosong()` | 5 | `void` — memanggil `cariIndexKosong()` di dalamnya; jika hasilnya bukan `-1`, isi index
tersebut |
| `hitungRataRata()` | 6 | `double` |

**Rumus imputasi (pengisian data kosong) untuk time series:**

Untuk index hari `i` yang kosong:

```
suhuHarian[i] = ( suhuHarian[i-1] + suhuHarian[i+1] ) / 2
```

Ini adalah metode imputasi paling dasar untuk data deret waktu, yaitu **mengisi nilai yang hilang dengan rata-rata dari
titik data tetangga terdekatnya** (sebelum dan sesudah), dengan asumsi nilai yang hilang berada di antara dua kondisi
yang sudah diketahui dan tidak berubah drastis.

Data suhu (array di `main`)

Gunakan array berikut sebagai data suhu 7 hari (Hari 1 = index 0):

```java
double[] suhuHarian = { 30.4, 24.3, 26.8, -1.0, 31.4, 30.8, 32.9 };
```
```

Alur program di `main`

1. Siapkan array `suhuHarian` seperti di atas.
2. Buat object `PengolahSuhu` dari array `suhuHarian` tersebut. Tampilkan data awal (`tampilkanData()`), lalu panggil `cariIndexKosong()` dan tampilkan index yang ditemukan.
3. Jalankan `isiDataKosong()`.
4. Tampilkan data akhir yang sudah bersih (`tampilkanData()`) beserta rata-rata suhunya.
5. Tampilkan kembali isi array `suhuHarian` di `main` (bukan lewat object `PengolahSuhu`, melainkan variabel array yang sudah disiapkan di langkah 1). Amati bahwa array ini pun sudah berubah, padahal tidak pernah diubah langsung di `main`, lalu jelaskan sebabnya dalam beberapa kalimat sebagai komentar pada kode program.

Output yang diharapkan

```
=== Data Suhu Awal ===
Hari 1 : 30.4°C
Hari 2 : 24.3°C
Hari 3 : 26.8°C
Hari 4 : (kosong)
Hari 5 : 31.4°C
Hari 6 : 30.8°C
Hari 7 : 32.9°C

Index hari kosong (dimulai dari 0): 3

=== Data Suhu Setelah Pengisian ===
Hari 1 : 30.4°C
Hari 2 : 24.3°C
Hari 3 : 26.8°C
Hari 4 : 29.1°C
Hari 5 : 31.4°C
Hari 6 : 30.8°C
Hari 7 : 32.9°C

Rata-rata : 29.39°C

Isi array suhuHarian di main setelah isiDataKosong() dijalankan:
[30.4, 24.3, 26.8, 29.1, 31.4, 30.8, 32.9]
(ikut berubah: constructor menyimpan referensi array yang sama)
```

Yang perlu diperhatikan saat mengerjakan

- Amati baik-baik dua baris terakhir pada output di atas. Array `suhuHarian` yang dicetak itu adalah variabel di `main`, bukan diambil lewat object `PengolahSuhu` — dan nilainya sudah berubah, padahal `main` tidak pernah memanggil operasi pengubahan itu sendiri. Ini terjadi karena array bertipe **reference**: constructor `PengolahSuhu` menyimpan array yang sama persis (bukan menyalinnya), sehingga perubahan yang dilakukan `isiDataKosong()` pada object otomatis terlihat juga dari sisi `main`. Jelaskan hal ini dalam beberapa kalimat sebagai komentar pada kode program.
- Nilai penanda `-1.0` harus ditulis satu kali saja sebagai class field, lalu dipakai oleh setiap method yang membutuhkannya. Jangan mengetik ulang angka `-1.0` di dalam method.
- Rumus imputasi mengasumsikan hari sebelum dan sesudah hari yang kosong tersedia sebagai data asli (bukan ikut kosong). Data pada soal ini sudah dijamin memenuhi asumsi tersebut.

Jawaban :

```
// nama file: PengolahSuhu.java

        public class PengolahSuhu {
            private double[] suhuHarian;
            public static final double NILAI_KOSONG = -1.0;

            public PengolahSuhu(double[] suhuHarian) {
                this.suhuHarian = suhuHarian;
            }

            public void tampilkanData() {
                for (int i = 0; i < this.suhuHarian.length; i++) {
                    if (this.suhuHarian[i] == NILAI_KOSONG) {
                        System.out.println("Hari " + (i + 1) + " : (kosong)");
                    } else {
                        // Diubah dari "°C" menjadi " C"
                        System.out.println("Hari " + (i + 1) + " : " + this.suhuHarian[i] + " C");
                    }
                }
            }

            public int cariIndexKosong() {
                for (int i = 0; i < this.suhuHarian.length; i++) {
                    if (this.suhuHarian[i] == NILAI_KOSONG) {
                        return i;
                    }
                }
                return -1;
            }

            public void isiDataKosong() {
                int indexKosong = cariIndexKosong();
                if (indexKosong != -1 && indexKosong > 0 && indexKosong <
        this.suhuHarian.length - 1) {
                    this.suhuHarian[indexKosong] = (this.suhuHarian[indexKosong - 1] +
        this.suhuHarian[indexKosong + 1]) / 2.0;
                }
            }

            public double hitungRataRata() {
                double total = 0.0;
                for (double suhu : this.suhuHarian) {
                    total += suhu;
                }
```

```
            return total / this.suhuHarian.length;
        }
}
```

```
// nama file : Main.java

    import java.util.Arrays;

    public class Main {
        public static void main(String[] args) {
            double[] suhuHarian = { 30.4, 24.3, 26.8, -1.0, 31.4, 30.8, 32.9 };

            PengolahSuhu pengolah = new PengolahSuhu(suhuHarian);

            System.out.println("=== Data Suhu Awal ===");
            pengolah.tampilkanData();
            System.out.println();

            int indexKosong = pengolah.cariIndexKosong();
            System.out.println("Index hari kosong (dimulai dari 0): " + indexKosong);
            System.out.println();

            pengolah.isiDataKosong();

            System.out.println("=== Data Suhu Setelah Pengisian ===");
            pengolah.tampilkanData();
            System.out.println();

            // Diubah dari "%.2f°C\n\n" menjadi "%.2f C\n\n"
            System.out.printf("Rata-rata : %.2f C\n\n", pengolah.hitungRataRata());

            System.out.println("Isi array suhuHarian di main setelah isiDataKosong()
    dijalankan:");
            System.out.println(Arrays.toString(suhuHarian));
        }
}
```

Output :

```
=== Data Suhu Awal ===
Hari 1 : 30.4 C
Hari 2 : 24.3 C
Hari 3 : 26.8 C
Hari 4 : (kosong)
Hari 5 : 31.4 C
Hari 6 : 30.8 C
Hari 7 : 32.9 C

Index hari kosong (dimulai dari 0): 3

=== Data Suhu Setelah Pengisian ===
Hari 1 : 30.4 C
Hari 2 : 24.3 C
Hari 3 : 26.8 C
Hari 4 : 29.1 C
Hari 5 : 31.4 C
Hari 6 : 30.8 C
Hari 7 : 32.9 C

Rata-rata : 29,39 C

Isi array suhuHarian di main setelah isiDataKosong() dijalankan:
[30.4, 24.3, 26.8, 29.1, 31.4, 30.8, 32.9]
```

Penjelasan :

File PengolahSuhu.java berisi kelas PengolahSuhu dengan instance field private double[] suhuHarian dan class field public static final double NILAI_KOSONG = -1.0. Objek dibuat lewat constructor menggunakan keyword this. Kelas ini memiliki method tampilkanData() untuk mengecek data kosong, cariIndexKosong() untuk mencari indeks -1.0, isiDataKosong() dengan rumus (suhuHarian[i-1] + suhuHarian[i+1]) / 2.0, serta hitungRataRata() untuk menghitung rata-rata suhu.

File Main.java menginisialisasi array double[] suhuHarian dengan nilai -1.0 di indeks ke-3. Program membuat objek pengolah, memanggil tampilkanData() dan cariIndexKosong(), lalu menjalankan isiDataKosong() untuk memperbarui data hilang serta mencetak rata-ratanya lewat hitungRataRata().

Di akhir, program mencetak ulang array suhuHarian di Main menggunakan Arrays.toString(suhuHarian). Nilai array di Main ikut berubah karena array bertipe reference (Pass by Reference), sehingga variabel di Main dan di dalam objek merujuk ke alamat memori yang sama.

D. Kesimpulan

Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan bahwa penerapan konsep dasar Pemrograman Berorientasi Objek seperti class, object, field, dan method memungkinkan program pengolahan data suhu dibangun secara terstruktur dan terenkapsulasi. Penggunaan class field bertipe static final sangat efektif untuk menetapkan konstanta bersama seperti penanda data kosong (-1.0), sedangkan instance field digunakan untuk menyimpan data spesifik milik masing-masing objek. Selain itu, pemanfaatan keyword this memudahkan pembedaan antara instance field dan parameter constructor yang memiliki nama identik.

Praktikum ini juga membuktikan penerapan konsep Pass by Reference pada tipe data non-primitif seperti array. Ketika array suhu harian dilewatkan ke dalam constructor objek, variabel di dalam objek menerima alamat memori yang sama dengan variabel array di kelas utama. Akibatnya, proses imputasi data yang dilakukan oleh method objek secara otomatis mengubah nilai array asal yang ada di method main. Hal ini memperjelas perbedaan mendasar antara pengiriman parameter berbasis nilai (Pass by Value) dan berbasis referensi (Pass by Reference) dalam pemrosesan data Java.